Computers & Security 157 (2025) 104567

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full Length Article

High-trigger fuzz testing for microarchitectural speculative execution vulnerability

Chuan Lu, Senlin Luo, Limin Pan *

Information System and Security & Countermeasures Experimental Center, Beijing Institute of Technology, 100081, China

ARTICLE INFO

Keywords:
Speculative execution
Speculative leak
Speculative Execution Vulnerability
Microarchitectural Vulnerability

ABSTRACT

Microarchitectural speculative execution vulnerabilities can be utilized to steal private information and even bypass some defensive programming measures in the code. The difficulty in detecting this vulnerability is ensuring a high triggering frequency of speculative execution. However, existing methods randomly generate test programs with high uncertainty, which lack dependencies relationship between code lines required by speculative execution, resulting in low trigger rates of speculative execution. Meanwhile, some variables of the test input are randomly selected for mutation, but the selected variables tend to lack the correlation with execution paths, leading to low detection adequacy and convergence of collected information. Therefore, this paper proposes a **H**igh-**T**rigger Fuzz Testing for Microarchitectural **S**peculative **E**xecution **V**ulnerability (HT-SEV). HT-SEV constructs a register selectied model, which generates subsequent codes based on the data flow and real-time register distribution of generated code, establishing data dependencies between different code lines. Furthermore, bidirectional gradient mutation is proposed, which mines the correlation between inputs and the collected microarchitectural information to guide the mutation of inputs, achieving high coverage of path and diversity of detection information. Experimental results on multiple instruction subsets show that HT-SEV outperforms state-of-the-art related methods. This method innovatively defines data dependency relationship, capturing fine-grained code execution information.

1. Introduction

Following the initial disclosure of the Spectre (Kocher et al., 2019) attack in 2018, speculative execution vulnerabilities (Downfall, 2023; Kim et al., 2023) have been assigned 38 CVEs (Common Vulnerabilities and Exposures, CVE), and related research continues to disclose new vulnerabilities. These vulnerabilities exploit widely used speculative execution mechanisms in modern processors, such as branch speculation, indirect jump address speculation, return address speculation, etc. ConsequentlyAs a result, speculative execution vulnerabilities affect nearly all modern processors, which not only seriously threaten the security of existing CPUs (Central Processing Unit, CPU) and microarchitectures, but also can bypass several defensive programming measures in code. For instance, direct array access is vulnerable to out-of-bounds memory accesses, which can be mitigated through boundary checking, as shown in Fig. 1. When speculative execution is enabled, attackers can bypass boundary checks by speculatively accessing array elements. Unfortunately, most speculative execution vulnerabilities are identified through data analysis and manual experimental testing, which are time-consuming and labor-intensive processes.

Existing testing methods predominantly rely on white-box (Rostami et al., 2024; Hur et al., 2022; Rajapaksha et al., 2023) testing, which require extensive background knowledge and are limited in detecting only specific vulnerabilities or architectural. This inherent limitation significantly restricts the applicability and scope of these testing methods across diverse scenarios. For problems with white box testing, Oleksenko develops Revizor (Oleksenko et al., 2022), a method to automatically mine speculative execution vulnerabilities based on Guarnieri (Guarnieri et al., 2021). In each test, Recizor randomly generates a test program and a set of inputs that produce identical contract traces. It then employs cache-side channel attacks to collect hardware trace. A contract trace represents the microarchitectural information theoretically exposed by a test program and its input, while a hardware trace refers to the actual microarchitectural information collected during execution, such as data loading address and data storage address. When the contract traces are the same under different inputs, the test program's execution should theoretically produce identical hardware

* Corresponding author.
E-mail address: panlimin2016@gmail.com (L. Pan).

https://doi.org/10.1016/j.cose.2025.104567
Received 20 February 2025; Received in revised form 22 May 2025; Accepted 6 June 2025
Available online 7 June 2025

traces for those inputs. If the collected hardware traces differ, a speculative leak is identified. However, randomly generated test programs exhibit high uncertainty, leading to low triggering rates of speculative execution. Additionally, if a set of inputs fails to produce consistent contract traces, testing with such inputs becomes meaningless.

To address the above problems, Oleksenko (Oleksenko et al., 2023) modified Revizor in 2023. First, a speculation filter and an observation filter are introduced to filter test programs. Then, a contract-driven input generator is employed to produce inputs for test programs that generate identical contract traces. These modifications increased the testing speed by two orders of magnitude. However, the modified Revizor does not resolve the essential problem of random test program generation. The absence of dependency relationship between code lines makes it difficult to meet speculative execution requirements, resulting in a low trigger rate of speculative execution. Meanwhile, when a test fails to detect a speculative leak, the modified Revizor uses random mutations to alter certain input variables. Unfortunately, the selected variables tend to lack the correlation with execution paths and microarchitecture states, resulting in low detection adequacy and convergence of collected information.

Therefore, two critical challenges arise in black-box microarchitecture speculative execution leak testing.

Problem 1. how to raise the trigger rate of speculative execution?

Problem 2. how to mutate test program inputs to increase detection diversity and adequacy?

Data dependency is a critical condition for boosting the trigger rate of speculative execution and its vulnerabilities. To reduce delays caused by slow preemptive code, speculative execution mechanisms leverage specific predictors (such as Branch Predictor and Predictive Store Forwarding Predictor) to anticipate the data and control flow of the code, improving execution efficiency. Triggering speculative execution can be summarized into two conditions: (a) the existence of dependencies between code lines and (b) the existence of slow-executing code.

For control dependencies, when the execution of a control instruction slows down due to specific reasons, the processor speculates on the branch path. There are two primary reasons for the slow execution. Firstly, the execution of a control instruction depends on the results of preceding instructions, indicating that the control instruction has a data dependency with the preceding instructions. As shown in Fig. 1, the execution of line 3 requires the execution results of line 1 and 2. Secondly, an operand of the control instruction loads slowly, such as when loading data from memory. Achieving this second condition during program generation is particularly challenging, as it is heavily influenced by external factors such as cache states and prefetching techniques. However, the first condition can be satisfied by constructing data dependencies between code lines. Consequently, data dependencies also play a crucial role in enhancing the trigger rate of speculative execution based on control dependencies.

From an attacker's perspective, the presence of a data dependency between vulnerable code and preceding code is essential. If the critical codes responsible for triggering the vulnerability lacks a data dependency with the preceding code, these codes accesses data at a fixed memory address. In such cases, an attacker cannot probe the private information by fixed data access. Only when the code exhibits a data dependency with the previous codes can it access data at varying addresses. Moreover, fixed accesses do not produce varying cache access traces. As a result, multiple runs of the code generate identical cache access traces. For instance, the line 4 of code in Fig. 1 becomes $\{data = foo[0]\}$. This scenario cannot be classified as a speculative execution vulnerability.

In summary, data dependency is a key factor in triggering speculative execution and detecting speculative execution vulnerabilities. It is defined as follows:

Definition I (Data Dependency). Given two code lines asm_1 and asm_2, data dependency exists between asm_1 and asm_2 if there exists a variable α in asm_1 that can affect the execution result of asm_2.

Therefore, this paper proposes **H**igh-**T**rigger Fuzz Testing for Microarchitectural **S**peculative **E**xecution **V**ulnerability (HT-SEV), which aims to strengthen the data dependencies between codes while diversifying the collected microarchitectural information (i.e., hardware trace). This method strengthens the ability to discover speculative execution vulnerabilities. During the test program generation phase, HT-SEV constructs a register selected model based on the real-time register distribution and data flow. This model guides the establishment of data dependencies and is continuously updated as the code is generated. When a test program and its inputs fail to trigger a speculative leak, HT-SEV changes the inputs by bidirectional gradient mutation. This technique leverages the correlation relationship between inputs and the collected microarchitectural information to achieve high coverage of paths and diversity of detection information.

In summary, the contributions are as follows:

(1) A data dependency-driven test program generation method is developed, which designs a register selected model based on strong data dependency property of the speculative execution. The model guides the establishment of data dependencies between codes through formed data flows and real-time register distribution, matching the triggering requirements of triggering speculative execution and speculative leak.
(2) A probing critical condition bidirectional gradient mutation method is proposed, which mines the correlation relationship between inputs and the collected microarchitectural information to guide the mutation, achieving high coverage of path and diversity of detection.
(3) Extensive experiments demonstrate that HT-SEV has the optimal performance compared to state-of-the-art related methods. The novel data dependency and bidirectional gradient mutation provide a significant boost in detecting speculative leaks, avoiding wasting time on the same microarchitectural information.

The organizing of the paper is as follows: Section 2 discusses related work on the speculative leak test; Section 3 details the principles and steps of HT-SEV; Section 4 discusses the performance comparison between HT-SEV and comparison experiments under various conditions; Section 5 develops a discussion around the speculative leak test; Section 6 summarizes the paper.

```
1.  data = foo[i]
```

(a) direct access

```
1.  i = input[0];
2.  size = len(input);
3.  if (i < size) {
4.      data = foo[i];
5.  }
```

(b) add boundary checking

Fig. 1. Defensive programming measure.

2. Related work

2.1. Speculative execution attack gadget detection

In a speculative execution attack, an attacker typically uses a specific code module, known as a speculative execution attack gadget, to manipulate the processor into triggering speculative execution and forcing sensitive data into a stealable state. Early research primarily employed static feature matching techniques (Clifton, 2018; Pardoe, 2018; Open Source Security Inc 2018). oo7 (Wang et al., 2019) extracts the control flow graph of a binary file and applies static taint analysis to detect the gadgets. However, oo7′s detection results heavily depend on the completeness of the control flow graph. Additionally, the inherent over-tainting and under-tainting issues of static taint analysis can lead to false positives or false negatives.

To address the limitations of static analysis, subsequent studies have utilized symbolic execution (Fabian et al., 2022; Ponce-de-León and Kinder, 2022) and dynamic taint analysis (Oleksenko et al., 2020; Johannesmeyer et al., 2022).

2.1.1. Detection methods based on symbolic execution

Wang (Wang et al., 2025) proposed an evaluation framework specifically designed for branch predictors, which abstractly describes the 19 states and 53 operations. However, this shortcoming is further exacerbated by the fact that symbolic execution is naturally plagued by the path explosion problem, especially when combined with speculative execution path analysis. To alleviate this problem, Daniel proposed the Haunted RelSE (Daniel et al., 2021) method, which dynamically associates constraints on regular and speculative execution paths and utilizes the relational symbolic execution BINSEC/REL (Daniel et al., 2020) for detection. This method mitigates the path explosion problem caused by speculative execution paths, but does not resolve the fundamental limitations of symbolic execution.

2.1.2. Detection methods based on taint analysis

SpecTaint (Qi et al., 2021) performs system-level simulations of speculative execution and identifies speculative execution attack gadgets through predefined behavioral features. The method saves the current state before simulating each speculative execution path and restores the state to its prior condition upon completion of the simulation. However, the system-level simulation of speculative execution and state rollback imposes a significant time overhead. Subsequently, Teapot (Lin et al., 2024) addressed the inefficiency of SpecTaint but relies on static taint analysis.

Tol M C (Tol et al., 2021) argued that existing detection methods rely on handwritten rules, which struggle to cover the subtle differences of known speculative execution attack gadgets. Therefore, Tol M C proposed FastSpec based on the BERT (Bidirectional Encoder Representations from Transformers, BERT). However, the method only takes each function as a detection object and does not determine whether the detected gadgets are in the speculative execution path. To address this problem and the inefficiency of dynamic detection, SCR-Spectre (Lu et al., 2025) first utilizes dynamic exploration of conditional instructions and then statically analyzes all possible paths. This method avoids both redundant detection of speculative execution paths and eliminates the additional time overhead introduced by state rollback.

2.2. Microarchitectural speculative execution vulnerability detection

One of the challenges in mining microarchitectural speculative execution vulnerability is the lack of visibility into the microarchitectural state. Consequently, many methods utilize white boxes to solve the problem, but white box testing can only be applied until the processor design is complete. CheckMate (Trippel et al., 2018) determines whether the microarchitecture is vulnerable to specific categories of attacks. CheckMate utilizes microarchitectural happens-before graphs to capture the subtle sequencing and interspersing of hardware execution events during program execution. IntroSpectre (Ghaniyoun et al., 2021) is an RTL-based framework designed to detect Meltdown-type leaks. It addresses the challenge of lack of visibility into the microarchitectural state by integrating into the RTL (Register Transfer Level, RTL) design flow, thereby gaining access to the processor's internal state. Similarly, Fadiheh (Fadiheh et al., 2019) proposes to detect whether a covert channel exposes private information by performing model checking on the RTL model of a RISC-V CPU.

Compared to white-box testing, black-box testing (Gras et al., 2020; Weber et al., 2021) does not require insight into the specific design details of the processor and can be applied to commercial processors. Black-box testing is primarily divided into two categories: template-based variant testing and model-based relational testing.

2.2.1. Template-based variant testing

Moghimi (Moghimi et al., 2020) proposed a testing tool called Transynther, which mutates the basic blocks of existing Meltdown variants to generate and evaluate new Meltdown subvariants. SpeechMiner (Xiao et al., 2020) is a vulnerability assessment framework for known speculative execution vulnerabilities. The framework leverages covert channel techniques and differential testing to gain visibility into changes in the state of microarchitectural structures. However, this method is limited to detecting known vulnerabilities, making it challenging to uncover new speculative execution vulnerabilities.

2.2.2. Model-based relational testing

Scam-V (Nemati et al., 2020) combines symbolic execution, relational analysis, and automated program generation techniques for experimentation and verification. The experiments involve a randomly generated program and two equivalent inputs. Verification is achieved by executing the program and analyzing side channels to confirm the indistinguishability of the two inputs on the actual hardware. Buiras (Buiras et al., 2021) enhances the observation model of Scam-V for finer-grained observations. Ragab (Ragab et al., 2021) focuses on vulnerabilities based on MC (Machine Clearing), such as Floating Point MC, Self-Modifying Code MC, Memory Ordering MC, and Memory Disambiguation MC. However, this method is tested manually, similar to Li (Li and Gaudiot, 2021).

Guarinieri (Guarnieri et al., 2021) proposes Contract, a theoretical testing framework for detecting speculative execution vulnerabilities in microarchitecture. The Contract can specify what mitigations the defender can take and what microarchitectural information the attacker can obtain. Oleksenko (Oleksenko et al., 2022) proposes Revizor, a tool for detecting microarchitectural speculative violations in commercial black-box CPUs. Revizor automatically generates test cases (test programs and inputs) to identify violations in microarchitecture.

Oleksenko (Oleksenko et al., 2023) discovers that Revizor spends a significant amount of time on the test cases that fail to meet speculative execution requirements. To address this issue, Oleksenko develops filtering rules by defining test case validity, which reduces the percentage of invalid test cases and prevents the testing process from wasting time on invalid test cases. Hofmann (Hofmann et al., 2023) proposes the first CPU anomaly leakage model for testing black-box CPUs. Through experiments on four different x86 microarchitectures, Hofmann identifies three new speculative execution vulnerabilities related to non-canonical address storage, read-only memory storage, and divide-by-0.

In summary, the test programs of the existing method rely on random generation, which is difficult to satisfy the strong data dependency properties of speculative execution, resulting in low trigger rates of speculative execution. Additionally, when a test fails to mine a speculative leak, existing methods randomly select some variables of the program inputs for mutation. However, some variables of the inputs lack the correlation with execution paths and microarchitectural states, causing low detection adequacy and convergence of the

microarchitectural information. Therefore, a speculative leakage testing method that guarantees a high trigger rate of speculative execution and diverse vulnerability detection basis needs to be proposed.

3. Principle of HT-SEV

3.1. Framework

The core idea of this method is to generate the test programs based on the observed data dependency required to trigger speculative execution, ensuring a high triggering rate. Additionally, the specific targeted mutation guidance is provided according to the execution path and data flow of each test program, achieving high path coverage and diverse vulnerability detection.

The main contribution of this method is the proposal a **H**igh-**T**rigger Fuzz Testing for Microarchitecture **S**peculative **E**xecution **V**ulnerability (HT-SEV). This method designs a register selected model to guide the construction of data dependencies among different code lines, aligning with the strong data dependency required for speculative execution. Furthermore, HT-SEV introduces a novel bidirectional gradient mutation strategy, enabling targeted mutation guidance and diversified detection.

The principle of HT-SEV is shown in Fig. 2 and consists of four parts. The inputs of the whole process are the instruction subset and the registers subset, while the outputs are violations containing the test program and the inputs of the test program:

(1) Data Dependency-Driven Test Program Generation

This part first generates the control flow graph to ensure the correct control flow of the program. It then generates assembly code based on the instruction subset and register subset. Finally, a speculation filter and an observation filter eliminate the test programs that do not meet the requirements.

(2) Test Program Input Generate

This part generates a set of inputs for the test program which generates the same contract trace.

(3) Program execute and Information Collect

This part involves executing the generated test programs with their inputs and collecting microarchitectural information during execution.

(4) Violation Analyze

The collected microarchitectural information is to determine whether there is a violation. If a violation is detected, this part outputs the violation along with the corresponding program and inputs. If no violation is found, actions such as mutating the inputs or increasing the number of codes is taken to enhance diversity.

3.2. Data dependency-driven test program generate

Existing methods randomly select instructions and operands from the input space, which struggle to satisfy the strong data dependency required by speculative execution, resulting in low trigger rate of speculative execution.

HT-SEV emphasizes strengthening data dependencies between different code lines during test program generation, which ensure that speculative execution can be triggered with high probability to detect speculative leaks.

The schematic diagram of Data Dependency-Driven Test Program Generate is illustrated in Fig. 3. Procedure 1 outlines the detailed process. The first and second lines generate a control flow diagram for the test program to ensure the correct control flow, as shown in Fig. 4. The operands for the conditional branch instruction are determined in line 8.

Lines 3~11 describe assembly code generation process. First, the register selected model assigns an initial selected probability to each register in the register subset (line 3). Then, HT-SEV generates assembly code based on the current register selected probabilities (line 8). After generating the assembly code, the register selected model updates the current register distribution information, including frequency, step length, and register first occurrence information (line 9). Frequency *freq* refers to the number of times a register has appeared in the generated code. Step length *len* is the distance from the last code line containing the register to the code line that is being generated. First occurrence information refers to which code block a register first appears in and whether it requires an initial value. As shown in Fig. 5, the frequency of register *rax* is 3. The last code line containing *rax* is 4 and the code line being generated is 5. Thus, its step length is $len = 5 - 4 = 1$. The frequency of register *rdx* is 1 and its step length is 4. The minimum step length is 1, and the maximum value is len_{max}, where len_{max} is the number of code lines that have been generated. In Fig. 5, $len_{max} = 4$. If a register does not appear in the previously generated code, the step length is marked as len_{max}. The first occurrence information of the register *rax* is the line 1 of *block*0 and register *rax* doesn't require an initial value. Once the register

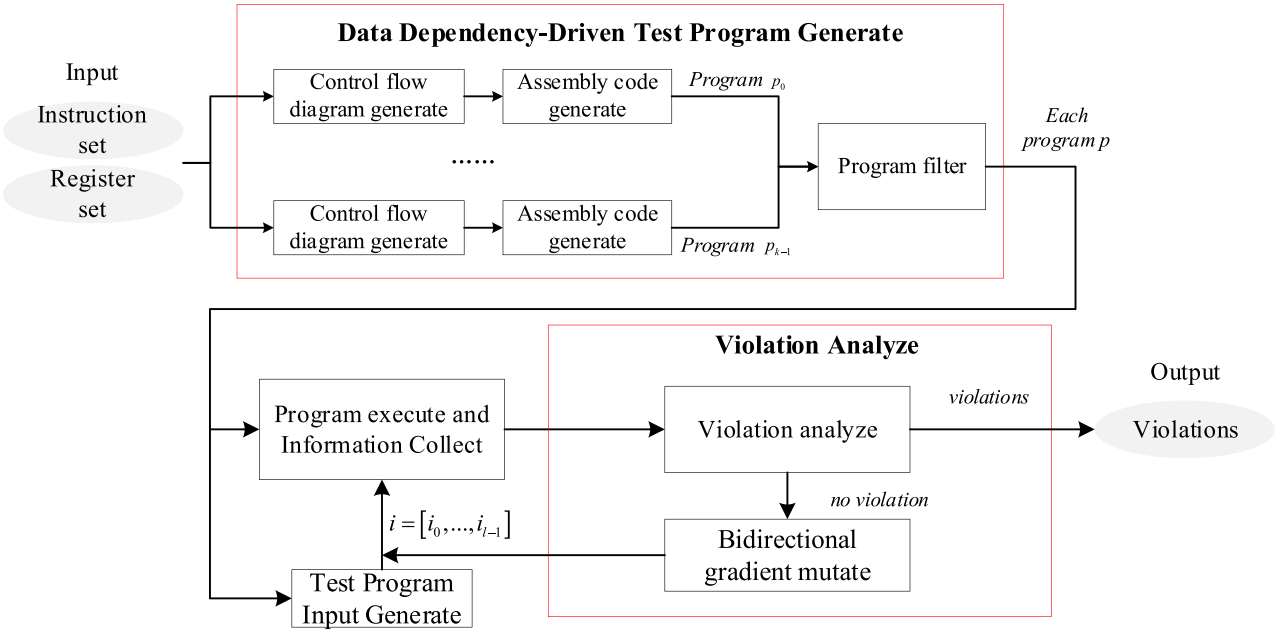

Fig. 2. The principle of HT-SEV.

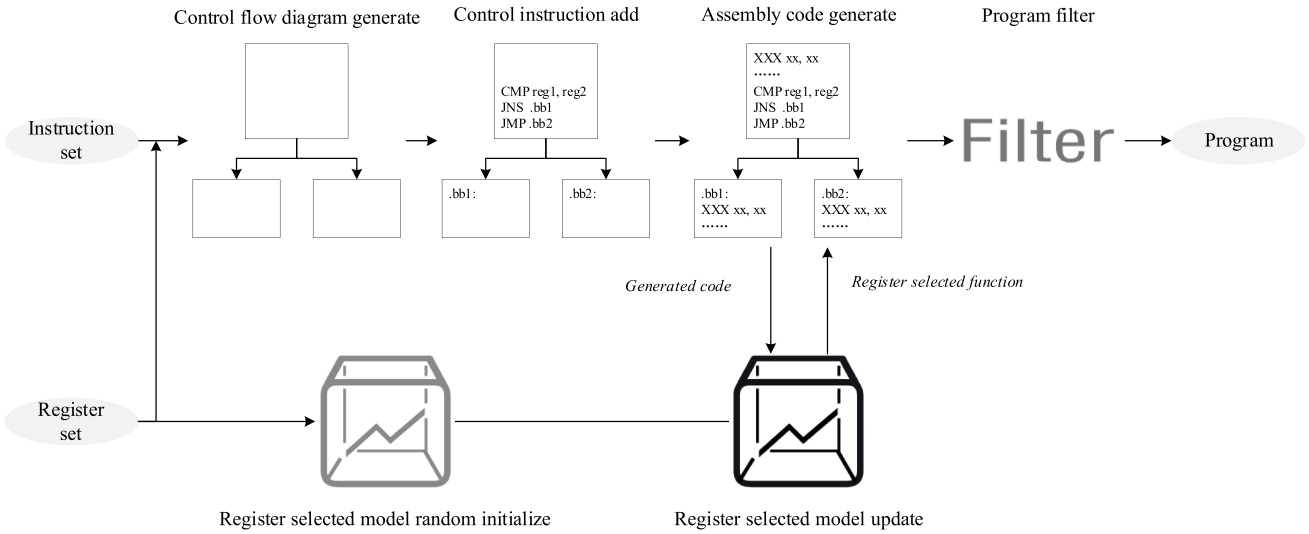

Fig. 3. The schematic diagram of Data Dependency-Driven Test Program Generate.

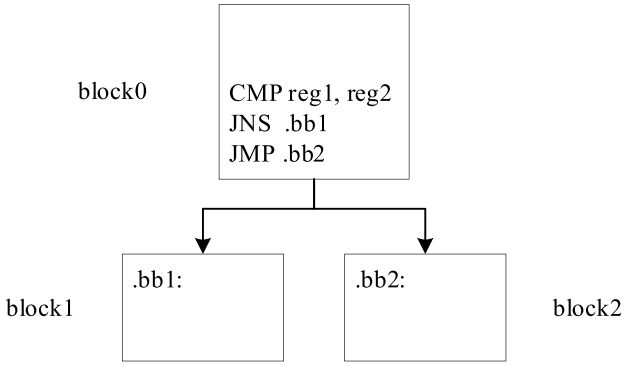

Fig. 4. Control flow diagram generation.

```
1 ADD  rax rdx
2 CMP  rax, 10
3 JNE  .END
4 MOV rax, [rbx]
5
```

Fig. 5. Register distribution information (frequency, step length, and register first occurrence information). When generating the line 5, the frequency of register ***rax*** is 3. The last code line containing ***rax*** is 4 and the code line being generated is 5. The step length of ***rax*** is ***len*** $= 5 - 4 = 1$. The frequency of register ***rdx*** is 1 and the step length is 4. The first occurrence information of the register ***rax*** is the line 1 of ***block***0 and register ***rax*** doesn't require an initial value.

distribution information is updated, the register selected model updates the register selected probability (line 10).

The key to constructing a test program lies in establishing data dependencies between different code lines. In line 3, each register is assigned an initial selected probability, and iterative adjustments of selected probabilities ensures that data dependencies are established effectively. In line 9, HT-SEV counts two critical factors for establishing data dependencies: frequency and step length. In a program, if a register appears more frequently, the probability of it forming data dependencies with subsequent code lines increases. Therefore, the register selected probability should be positively correlated with frequency. Conversely, step length exhibits an inverse relationship. Speculative execution is designed to address processor waiting caused by data dependencies. If the distance between two code lines asm_a and asm_b which have data dependency is too long, triggering speculative execution is difficult. When asm_a executes slowly, the processor may proceed with subsequent code. However, due to the long distance between asm_a and asm_b, asm_a may have already completed execution before asm_b is executed speculatively. Thus, the register selected probability should be negatively correlated with step length. Consequently, the register selected probability can be updated using the formula:

$$P_i^{k+1} = P_i^k * \frac{freq}{len} \tag{1}$$

where i is a register and k is the number of the code line. P_i^k is the selected probability of register i when generating the k-th code line. *freq* is the frequency. *len* is the step length.

However, this treatment leads to an increasing selected probability for registers with high frequency, resulting in code that predominantly uses a limited set of registers. To address this issue, the formula for updating the register selected probability incorporates a mutation term:

$$P_i^{k+1} = P_i^k * \frac{freq}{len} + \alpha * rand\left[0, \max\left(P_0^k, \ldots, P_{m-1}^k\right)\right] \tag{2}$$

where *rand* is a random function with a value range of $\left[0,\ \max\left(P_0^k, \ldots, P_{m-1}^k\right)\right]$. α is the mutation coefficient with a value range of $[0,\ 1]$. After updating each register selected probability, normalization is applied to ensure consistency:

$$S_i^{k+1} = \frac{P_i^{k+1}}{\sum_{j=0}^{m-1} P_j^{k+1}} \tag{3}$$

During test program generation, the selected probability of each register is dynamically adjusted following the process. For each generated code line, HT-SEV tracks register distribution information (line 9) and updates the register selected probability (line 10).

After generating test programs, Procedure 1 uses the same speculation filter and observation filter as in Oleksenko (Oleksenko et al., 2023). The speculation filter eliminates the test program that cannot trigger speculative execution. HT-SEV leverages two hardware performance counters: UOPS_ISSUED.ANY and UOPS_RETIRED.SLOTS. UOPS_ISSUED.ANY measures recovery cycles from both machine clears and branch misprediction, while UOPS_RETIRED.SLOTS counts the number of transient micro-operations. If either of these hardware performance counters is a non-zero value, the test program is confirmed to have triggered speculative execution. The observation filter ensures that the test programs have observable hardware traces of speculative execution. Specifically, the test program p adds the serialization fence *lfence* to form p_{ser}. With the same inputs, HT-SEV collects hardware traces for both p and p_{ser}. If the two traces differ, the test program p is determined to have an observable hardware trace. The methodology for hardware trace collection is detailed in Section 3.4.

Implementation This part describes the implementation of data dependency-driven test program generation, and includes an example for clarity. The overall procedure is as follows:

Step 1: Generate control flow (line 1–2).

The process begins by generating three code blocks, which are combined into a two-branch control flow graph. These blocks are labeled *block*0, *block*1, and *block*2, as shown in Fig. 4. *block*0 is augmented with a conditional instruction and corresponding control instructions. The operands for the conditional instruction are determined in Step 2.

Step 2: Generate assembly code (line 3–11).

Following the control flow construction, HT-SEV generates specific assembly instructions. This process consists of several stages:

(1) *Instruction Count Allocation:* HT-SEV randomly determines the number of instructions for each block based on a predefined instruction limit—32 instructions in this experiment.
(2) *Register Selected Model Initialization:* HT-SEV initializes the register selected model, and assign a selected probability to each register such that summing these probabilities equals 100 %.
(3) *Instruction Generation:* HT-SEV first generates a random assembly instruction. Assuming that the assembly instruction requires n registers, HT-SEV outputs n registers according to the register selected model and fills them into the assembly instruction sequentially. Once one block is complete, HT-SEV proceeds to the next until the limit of three blocks is reached.
(4) *Model Update:* After generating each instruction, HT-SEV updates the register selected model. HT-SEV firstly counts the register distribution information of the generated code, as shown in Fig. 5. Second, HT-SEV updates the selected probability of each register according to the formula (2) and (3).

Step 3: Correction and Compilation.

To avoid program errors, HT-SEV makes necessary essential adjustments to the generated test program: (1) modifying the memory address and restricting it to the dedicated memory area; (2) modifying the operands of division to avoid division by zero. Finally, HT-SEV compiles the test program into a binary file.

Example. Consider the generation of the first line {ADD *rax*, *rdx*} in Fig. 5, assuming input register $reg_{subset} = \{rax, rbx, rcx, rdx\}$. (1) Assign a selected probability to each register $\{rax = 40\%, rbx = 30\%, rcx = 20\%, rdx = 10\%\}$. (2) Randomly generate an instruction $\{\text{ADD } reg_0, reg_1\}$. (3) The register selected model selects two registers $reg_0 = rax$ and $reg_1 = rdx$. (4) HT-SEV updates the register distribution information and the register selected probability, as shown in Table 1.

3.3. Test program input generate

This part focuses on generating a set of test program inputs that produce the same contract trace. Revizor (Oleksenko et al., 2022) randomly generates inputs for the test program, but this method results in a low percentage of the valid inputs that produce the same contract traces. To solve this problem, the paper (Oleksenko et al., 2023) uses a Contract-driven Input Generator. However, the paper (Oleksenko et al., 2023) cannot generate the inputs that generates the same contract trace if all the registers requiring initial values precede the conditional branch instruction. Changing any part of the input (i.e., changing the initial value of any register in the input) would changes the contract trace.

During the test program generation phase, HT-SEV detects whether such a situation arises. If the register distribution meets a critical condition for this situation, HT-SEV adjusts the selected probability of each register to prevent it from occurring. Furthermore, HT-SEV can quickly locate the variables that do not affect contract trace of the current path based on the information of register occurrences and subsequently generates test program inputs.

As shown in Fig. 6, Example 1 is the same as the example provided in the paper (Oleksenko et al., 2023). When the input is $i = \{rax = 20, rbx = 5\}$, the execution path of this program proceeds from *block*0 to *block*2. Although register *rbx* is present in the {*MOV rbx*, [*rax*]} within *block*2, its initial value does not affect contract trace under the current condition. Therefore, the registers in the input that do not affect the contract trace must satisfy the following condition: the register does not require an initial value at its first occurrence in the current execution path.

In line 9 of Procedure 1, the register selected model record register first occurrence information. Using this information, HT-SEV can determine which part do not affect contract trace and quickly generate eligible inputs. However, if the registers *rbx* first appears in *block*0, changing either register *rax* or *rbx* cannot generate the same contract trace, as demonstrated in the example 2 in Fig. 6. To solve this problem, the register selected model adjusts the register selected probability based on the register first occurrence information. Let the subset of registers contain m registers, and the adjustment rule is as follows:

(1) When $m-2$ registers all appear in *block*0, then the selected probability of the remaining 2 registers is 0 in *block*0,
(2) When none of the registers in the two branches is the first occurrence and requires an initial value, the selected probability of the remaining registers that have not yet appeared is doubled, and these registers are preferentially set as source operands.

3.4. Program execute and information collect

The primary objective of this section is to collect the hardware trace.

Table 1
Example of updating the register selected model. ***block***0 refers to the block where the register initially appears. The label ***T*** signifies that the register requires an initial value, whereas ***F*** denotes that no initial value is required.

Register	Selected probability		Register distribution information of the generated code			
	Before	After	Frequency	Step length	First occurrence information	
rax	40 %	45 %	1	1	*block*0	*T*
rbx	30 %	25 %	0	1	–	*F*
rcx	20 %	15 %	0	1	–	*F*
rdx	10 %	15 %	1	1	*block*0	*T*

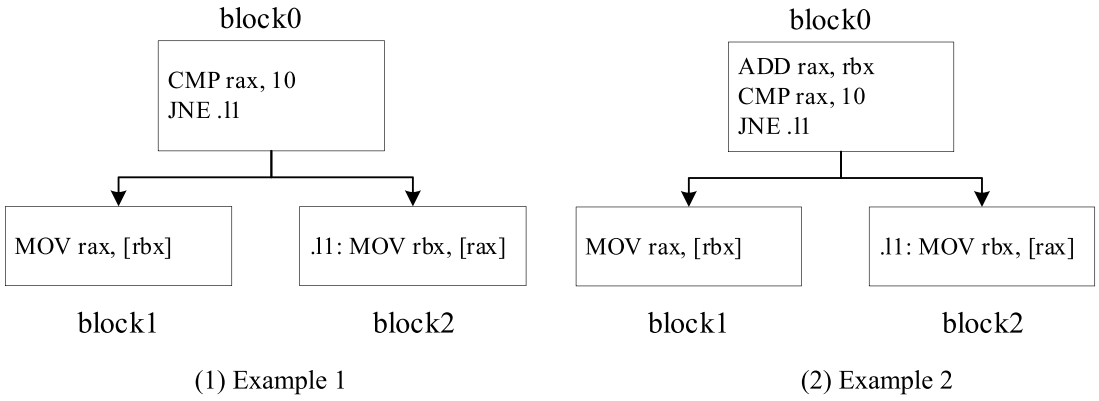

Fig. 6. Test program input generate.

As defined in the paper (Guarnieri et al., 2021), this part involves gathering the data loading address, data storage address, etc. Due to the challenges in obtaining precise addresses for data load and storage, this section uses the cache to determine which data is accessed. During the execution of the test program, a hardware trace of the test program and an input is generated, in conjunction with a cache-side channel attack that probes which cache set has been accessed. After collecting the hardware traces for each test program input, these hardware traces are compared. If a hardware trace of a specific input is inconsistent with those of the other inputs, then a violation exists. Such a treatment matches the actual conditions, and the attacker also uses the cache-side channel attack (Tromer et al., 2010; Gruss et al., 2016; Disselkoen et al., 2017) to obtain the data that is exposed because of the speculative execution. HT-SEV supports various cache-side channel attacks, such as Prime+Probe (Osvik et al., 2006), Flush+Road (Yarom and Falkner, 2014), and Evict+Road (Gruss et al., 2015).

As illustrated in Table 5, this manuscript enumerates speculative execution vulnerabilities with CVE numbers. In most cases, speculative execution attacks must be combined with other techniques to extract private information. This creates an opportunity to collect hardware traces using cache-side channel attacks. However, a small subset of speculative execution vulnerabilities cannot directly leverage cache-side channel attacks. However, these vulnerabilities are limited to specific processor architectures. For instance, the Zenbleed vulnerability is exclusive to processors with the AMD Zen2 architecture.

During the collection of microarchitectural information, hardware trace is also affected by the microarchitectural context. In different microarchitectural contexts $\mu_1 \neq \mu_2$, for the same test program p and the input i_1, i_2 that generates the same contract traces, i.e., $Contract(p,\ i_1) = Contract(p,\ i_2)$, the hardware traces may differ:

$$Measure(p,\ i_1, \mu_1) \neq Measure(p, i_2, \mu_2) \tag{4}$$

To address this issue, HT-SEV implements several measures to mitigate the impact of microarchitectural contexts. First, HT-SEV generates an input sequence that produces the same contract traces and inserts i_1, i_2 into this sequence. Inputs at the same position in this sequence can be approximated as the same microarchitectural context. The input sequence is sequentially fed into the test program p, and hardware traces $Measure(p,\ i_1, \mu_1)$ and $Measure(p, i_2, \mu_2)$ are collected. Next, the positions of i_1 and i_2 are exchanged, and the hardware traces $Measure(p,\ i_2, \mu_1)$ and $Measure(p, i_1, \mu_2)$ are collected. Finally, by comparing the four hardware traces across the two microarchitectural contexts, HT-SEV determines whether a speculative execution violation is occurred.

3.5. Violation analyze

The existing methods (Oleksenko et al., 2022; Oleksenko et al., 2023) adopt random mutations to change the test programs inputs without triggering speculative leaks, while some partial of inputs lack the correlation with execution paths and microarchitecture states, leading to low detection adequacy and convergence of collected information.

Example. As shown in Fig. 7, the input to this test program is $i = \{rax = 15, rbx = 10\}$. At this point, the execution path is 1→2→4. Papers (Oleksenko et al., 2022) and (Oleksenko et al., 2023) randomly select *rax* or *rbx* for mutation. If the random variable is *rbx*, the execution path cannot change. Additionally, if *rax* changes to rax = 20, the execution path remains unchanged. Consequently, random mutation struggles to accurately altering the execution path, thereby reducing detection efficiency and adequacy.

Traditional fuzzing tests (Lu et al., 2024; Zhu et al., 2022) typically improve test coverage through multiple tests, execution feedback, and complex mutation strategies. However, in speculative leak testing scenarios, such complex patterns are not practical enough. Compared with traditional fuzzy testing, speculative leak testing generates relatively more simple test programs. In the experimental part of this paper, the test program constructed by HT-SEV contains at most only 32 instructions, and 100,000 test programs are generated for each vulnerability test. Complex test patterns generally demand a higher number of test iterations and incur longer time overheads, whereas simpler program structures can achieve the desired results more directly and efficiently. Therefore, in speculative leak testing, the design of the mutation module should focus on simplicity and efficiency.

The main idea of this section is to explore the differential effects of different part on the execution path, thus providing efficient and accurate mutation guidance.

The process is shown in Procedure 2 and Fig. 8. HT-SEV identifies a violation when a program p exhibits inconsistent hardware traces across different inputs $i = [i_0, i_1, \ldots, i_{l-1}]$ (line 1). When a test involving a test program p and a set of inputs $i = [i_0, i_1, \ldots, i_{l-1}]$ does not reveal a violation, HT-SEV checks whether all instructions of the test program p are tested (line 6). If the test coverage $cover_{max}$ is not maximized, HT-SEV changes the inputs using a bidirectional gradient mutation.

Implementation The core of bidirectional gradient mutation is to rapidly maximize the coverage $cover_{max}$. Consequently, bidirectional gradient mutation employs as few and simple operations as possible rather than the existing complex framework. The specific rules are as follows:

Step 1: Select a mutation variable: Based on the register first

```
1 CMP rax, 10
2 JNE .l1
3 MOV rax, rbx
4 .l1: MOV rbx, rax
```

Fig. 7. Examples of random mutation. the input to this test program is $\boldsymbol{i} = \{\boldsymbol{rax} = 15, \boldsymbol{rbx} = 10\}$. At this point, the execution path is 1→2→4.

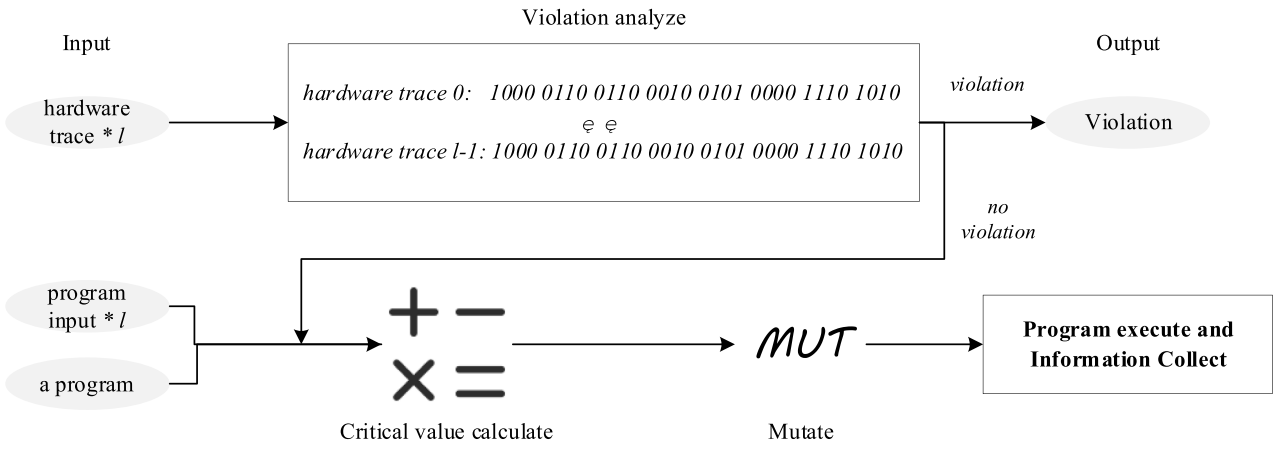

Fig. 8. The schematic diagram of Violation Analyze.

occurrence information, select a register that first appears before the conditional branch instruction as the mutation variable x. The original value of x is denoted as x_0.

Step 2: Extract the result variable: Extract the operand of the conditional branch instruction as the result variable y. If the conditional branch instruction has only one operand, $y = operand$. If there are two operands, $y = operand_1 - operand_2$. The original value of y is denoted as y_0.

Step 3: Upward mutation: Mutate the variable x upwards to obtain $x_1 = x_0 + \alpha$ ($\alpha > 0$) and execute the test program to obtain y_1. If the result of the conditional branch instruction changes (i.e., the execution path changes), proceed to step 7.

Step 4: Downward mutation: Mutate the variable x downwards to obtain $x_2 = x_0 - \beta$ ($\beta > 0$) and execute the test program to get y_2. If the path changes, proceed to step 7.

Step 5: Calculate critical value: Use (x_1, y_1) and (x_2, y_2) to construct a linear function and calculate the critical value of the path change ($y = 0$). The critical value of the mutation variable is recorded as x_3. HT-SEV executes the test program by x_3. If the execution path changes, proceed to step 7. When the linear function cannot be constructed (e.g. $y_1 = y_2$), proceed to step 6.

Step 6: Exchange branch addresses: Exchange the addresses of the two conditional branches to generate a new test program if the path does not change.

Step 7: *Re*-enter testing phase: The newly generated test program and inputs re-enter the phase of Program execute and Information Collect (Section 3.4).

Example. The input to this test program is $i = \{rax = 15, rbx = 10\}$, as shown in Fig. 9(a). Output modules are inserted at the head of both branches for determining whether to execute the respective branch, as shown in Fig. 9(b). The execution path of this test program is 1→2→4.

(1) Based on the register first occurrence information, *rax* is selected as the mutation variable x (Step 1).
(2) The result variable calculation module is inserted at the conditional judgment and the result variable is set to $y = rax - 10$, as shown in Fig. 9(b) (Step 2). Within this module, HT-SEV selects a register that do not influence the result of the conditional judgment for the calculation of the result variable based on the register first occurrence information.
(3) Mutation variable x is mutated upward to $\{rax = 18, rbx = 10\}$ and the execution path is probed (Step 3). At this stage, the result variable is $y_1 = 18 - 10 = 8$, and the execution path is unchanged.
(4) Mutation of mutation variable x is mutated downward to $\{rax = 12, rbx = 10\}$ and the execution path is probed (Step 4). At this point, the result variable is $y_2 = 12 - 10 = 2$, and the execution path remains unchanged.
(5) The key value of the result variable is calculated as $x_3 = 10$, and the execution path is probed (Step 5). At this juncture, the result variable is $y_3 = 10 - 10 = 0$, altering the execution path to 1→2→3.
(6) HT-SEV updates the input value to $i = \{rax = 10, rbx = 10\}$ and re-enters the Program execute and Information Collect phase (Step 7).

```
1 CMP rax, 10
2 JNE .l1
3 MOV rax, rbx
4 .l1: MOV rbx, rax
```

(a) the test programs that require mutation

```
  MOV rbx, rax
  SUB rbx, 10
  output module: rbx
1 CMP rax, 10
2 JNE .l1
  output module: "block 1"
3 MOV rax, rbx
 .l1:
  output module: "block 2"
4 MOV rbx, rax
```

(b) the test program in mutation

Fig. 9. Example of a bidirectional gradient mutation. The input to this test program is $\boldsymbol{i} = \{\boldsymbol{rax} = 15, \boldsymbol{rbx} = 10\}$. The result variable calculation module is highlighted in red, while the output module is labeled in green. Within the result variable calculation module, HT-SEV selects a register that do not affect the outcome of the conditional judgment based on the register first occurrence information, for use in calculating the result variable $\boldsymbol{y}$.

4. Experiments and analysis

4.1. Purpose

The 6 experiments in this section comprehensively evaluate the performance of HT-SEV. Additionally, HT-SEV is compared with 2 state-of-the-art methods, Revior (2022) (Oleksenko et al., 2022) and Revior-S (2023) (Oleksenko et al., 2023). The 6 experiments contain 4 comparison experiments and 2 ablation experiments.

The 4 comparison experiments are described below:

(1) Exp. 1: Percentage of Valid Test Programs. The experiment employs speculation and observation filters to evaluate the test programs generated by each method. The speculation filter eliminates test programs that fail to trigger speculative execution, while the observation filter ensures that the test programs produce observable hardware traces of speculative execution. The first filter represents the ability of the test program to trigger the speculative execution, whereas the second filter verifies that the speculative execution accesses data distinct from normal execution, which can be exploited by an attacker. Each method generates an equal number of test programs, and the percentage of programs that pass both filters is compared.
(2) Exp. 2: Number of Violations. This experiment evaluates the ability of each method to detect speculative leaks under the same conditions. In this experiment, each method generates the same number of test programs and test program inputs.
(3) Exp. 3: Detection Time. Detection time refers to the time for detecting a violation. Detection time is a comprehensive metric that is related to the total testing time and the number of violations. This metric reflects the detection speed of each method.
(4) Exp. 4: Valid Input generation. A valid input is defined as at least one other input that generates the same contract trace. This experiment tests the percentage of valid test program inputs generated by each method, and demonstrate the superiority of generating test program inputs by register first occurrence information. Meanwhile, this experiment verifies that the contract-driven input generation methods cannot generate valid test program inputs in some cases (as detailed in Section 3.3).

The 2 ablation experiments are described below:

(5) Exp. 5: Ablation Experiment 1. This experiment evaluates the performance of the Data Dependency-Driven Test Program Generation (Section 3.2) in facilitating the detection of speculative leaks. In this experiment, the data dependency-driven test program generation method in HT-SEV is replaced with random generation, while all other parts remain unchanged.

(6) Exp. 6: Ablation Experiment 2. Similar to Experiment 5, this experiment evaluates the performance of the Violation Analyze (Section 3.5). In this experiment, bidirectional gradient mutation in HT-SEV is replaced with random mutation and other parts remain unchanged.

4.2. Experimental setup

4.2.1. Data source

The 13 x86 instruction subsets are employed as datasets for evaluating each method because the x86 instruction set is widely used in various fields. Each instruction is categorized into different subsets depending on its type. To ensure the basic functionality of the test program, each instruction subset also contains 8 fundamental instructions (ADC, ADD, CMP, DEC, INC, NEG, SBB, SUB). The dataset excludes system instructions (e.g., SYSCALL), instructions incorrectly emulated by Unicorn (e.g., ROR), and control-flow instructions that are not contractually available (RET, CALL, and indirect jumps), which are not yet supported by the test framework. These x86 instruction subsets make up a comprehensive base x86–64 instruction set. Detailed information about each subset is provided in Table 6.

4.2.2. Experiment environment and condition

The experiments are all performed on an Intel(R) Core(TM) i9–9900 (Coffee Lake) CPU and the operating system is Ubuntu 18.04. The specific parameters are shown in Table 2.

4.2.3. Evaluation metrics

The 3 evaluation methods are adopted in this manuscript to evaluate the performance of each method. The number of violations evaluates the ability of each method to detect speculative leaks. The percentage of valid test program evaluates the ability of each method to generate valid test programs. Detection time evaluates the detection efficiency of each method.

Since the proposed method incorporates the cache-side channel attack determination vulnerability, a successful attack determines that a violation has been detected, which is consistent with the actual attack process. In addition, the proposed method employs some measures to eliminate the noise during testing and no false positives are found in the results. In the experiments of this manuscript, the testing for each vulnerability generates numerous test programs, and manually verifying each test program is impractical. Moreover, detecting whether a program contains code that triggers a speculative execution vulnerability belongs to the Detecting Speculative Execution Attack Gadget, as discussed in papers (Qi et al., 2021) and (Lu et al., 2025). Therefore, acknowledging the characteristics of speculative execution vulnerability and the constraints of realistic conditions, this manuscript adopts the number of violations instead of the traditional false-negative and false-positive as evaluation metrics. A more detailed explanation can be found in False Positives and False negatives in the Section 5 Discussion.

The specific definitions of each metric are as follows:

Number of violations is the count of violations detected in the given number of test programs and test program inputs. When a violation is detected in a test program's hardware trace, the number of violations is added by one. Each test program is counted only once.

Percentage of Valid Test Program is the percentage of generated test programs that pass the two filters, defined as follows:

$$p = \frac{n_p}{Num} \tag{5}$$

where n_p is the number of generated test programs and Num is the number that pass the filters.

Detection time is a time for detecting a speculative execution violation, defined as follows:

$$t = \frac{T}{num} \tag{6}$$

where T is the overall execution time and num is the number of violations detected.

4.2.4. Experiment procedure

Each experiment generates 100,000 test programs, each paired with 100 inputs. Each experiment evaluates four speculative execution vulnerabilities: Spectre V1 (CVE-2017–5753), LVI-Null (CVE-2020–0551), Spectre V4 (CVE-2018–3639), and V1-Var which belongs to the Spectre V1 class (Oleksenko et al., 2022). The specific configurations are shown in Table 3. HT-SEV, Revior (Oleksenko et al., 2022) and Revior-S (Oleksenko et al., 2023) all use the same configuration.

Table 2
Experiment environment.

Name	Describe
CPU RAM	Intel(R) Core(TM) i9–9900 @ 3.10 GHz 32 G RAM
System	ubuntu 18.04

Table 3
Experiment configurations.

Name	Configuration
Spectre V1	ISA subsets: cond, nop, bit, cmov, conv, dxfr, flag, setc, logi Target contract: CT-SEQ Spectre V4 patch enabled: True Microcode assists permitted: False
Spectre V4	ISA subsets: nop, bit, cmov, conv, dxfr, flag, setc, logi Target contract: CT-SEQ Spectre V4 patch enabled: False Microcode assists permitted: False
LVI-Null	ISA subsets: nop, bit, cmov, conv, dxfr, flag, setc, logi Target contract: CT-SEQ Spectre V4 patch enabled: True Microcode assists permitted: True
V1-Var	ISA subsets: nop, bit, cond, cmov, conv, dxfr, flag, setc, logi Target contract: CT-COND Spectre V4 patch enabled: True Microcode assists permitted: False

4.3. Comparison experiments

4.3.1. Exp. 1: percentage of valid test program

Fig. 10 illustrates the percentage of test programs generated that pass the two filters. Since both Revizor (Oleksenko et al., 2022) and Revizor-S (Oleksenko et al., 2023) rely on random generation, only one method is depicted in the figure. HT-SEV significantly outperforms the comparison method. The test programs generated by HT-SEV emphasize the construction of data dependency, which increases the probability of triggering speculative execution. Consequently, HT-SEV achieves a high rate of passing the first filter. Additionally, if a code line during speculative execution exhibits data dependency with the code line that triggers speculative execution, it utilizes speculative data, which differs from the data in normal execution. This results in a higher rate of passing the second filter.

4.3.2. Exp. 2: number of violations

The results for the number of violations are shown in Fig. 11. HT-SEV detected the highest number of violations, with an average increase of 26 % compared to Revizor-S (Oleksenko et al., 2023). When generating test programs, HT-SEV strengthens the data dependency between codes, ensuring that more test programs pass both filters. Test programs with strong data dependencies have a greater probability to trigger speculative execution and speculative violations. Additionally, in the Violation Analyze phase, HT-SEV increases the coverage of the test programs, further increasing the probability of triggering a speculative execution violation. In contrast, Revizor (Oleksenko et al., 2022) detects the fewest violations. Compared to Revizor-S (Oleksenko et al., 2023), Revizor (Oleksenko et al., 2022) generates a smaller percentage of the inputs that generate the same contract trace, resulting in a lack of test cases (test programs and test program inputs) that satisfy the test requirements to trigger speculative violations.

4.3.3. Exp. 3: detection time

Fig. 12 illustrates the comparison of testing time for each method. HT-SEV achieves an average increase of 17 % compared to Revizor-S[9], which is smaller than the increase observed in Experiment 1. This situation is primarily attributable to two reasons. First, HT-SEV generates a large percentage of test programs that pass through both filters, requiring more tests than Revizor-S[9] and resulting in a longer time to detect a violation. As HT-SEV detects more violations, it reduces the average time to detect a single violation instance. As a result, the overall testing time for HT-SEV is longer than Revizor-S[9]. In contrast, Revizor (Oleksenko et al., 2022) has the longest detection time because a significant amount of time is spent on meaningless test cases.

4.3.4. Exp. 4: input generation

This section compares the valid input percentages of the test programs, and the results are shown in Table 4. A valid input is defined as an input for which at least one other input generates the same contract trace. Revizor (Oleksenko et al., 2022), which uses random input generation, has the smallest percentage. In contrast, Revizor-S (Oleksenko et al., 2023), which uses a contract-driven input generator, achieves 99.97 %. As discussed in Section 3.3, Revizor-S (Oleksenko et al., 2023) cannot generate valid inputs if the registers requiring initial values are all present in block 1. HT-SEV avoids this issue during test program generation. Consequently, HT-SEV achieves the highest percentage of 100 %.

4.4. Albation experiments

4.4.1. Exp. 5: ablation experiment 1

This section evaluates the impact of the Data Dependency-Driven Test Program Generation (Section 3.2) on performance, and the results are shown in Fig. 13. HT-SEV-A serves as a comparison method, wherein the Data Dependency-Driven Test Program Generation is replaced by randomly generated test programs. HT-SEV increases by 20 % compared to HT-SEV-A. By leveraging register distribution information, HT-SEV constructs a register selected model which establishes data dependencies between code lines. The purpose of speculative execution aims to mitigate delays in subsequent execution caused by waiting for the previous code to complete. According to the previous discussion, data dependency is a critical factor in triggering speculative execution. Consequently, the Data Dependency-Driven Test Program Generation facilitates the triggering of speculative execution and makes it easier for the generated test program to trigger speculative violations.

4.4.2. Exp. 6: ablation experiment 2

Fig. 14 shows the results of the performance of the Violation Analyze

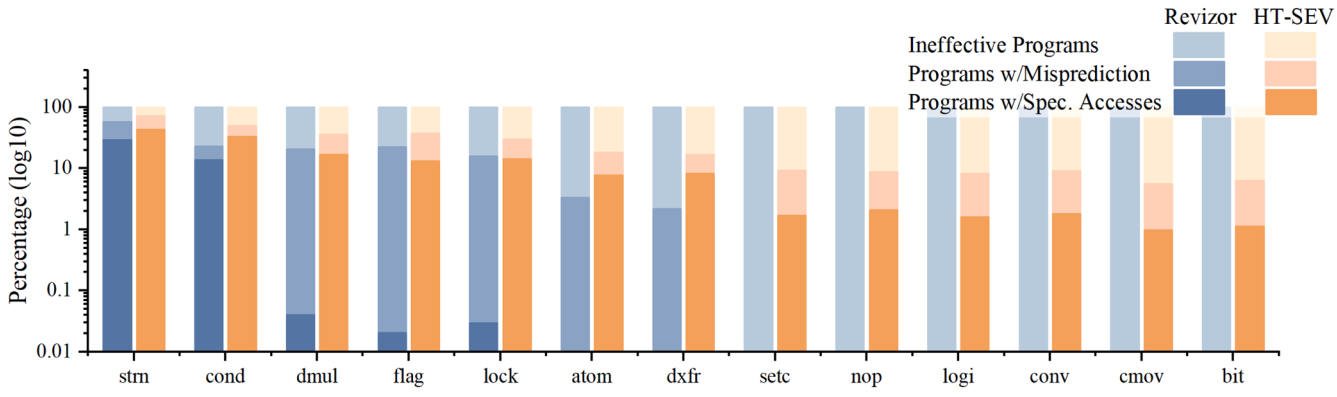

Fig. 10. Percentage of test programs generated that pass the two filters.

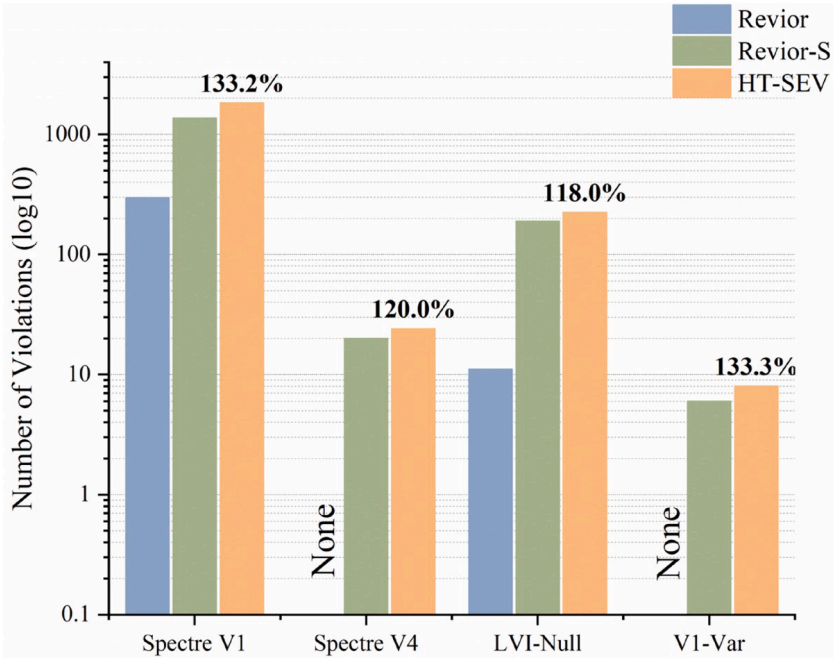

Fig. 11. Number of violations. The four percentages represent the proportion of HT-SEV compared to Revizor-S.

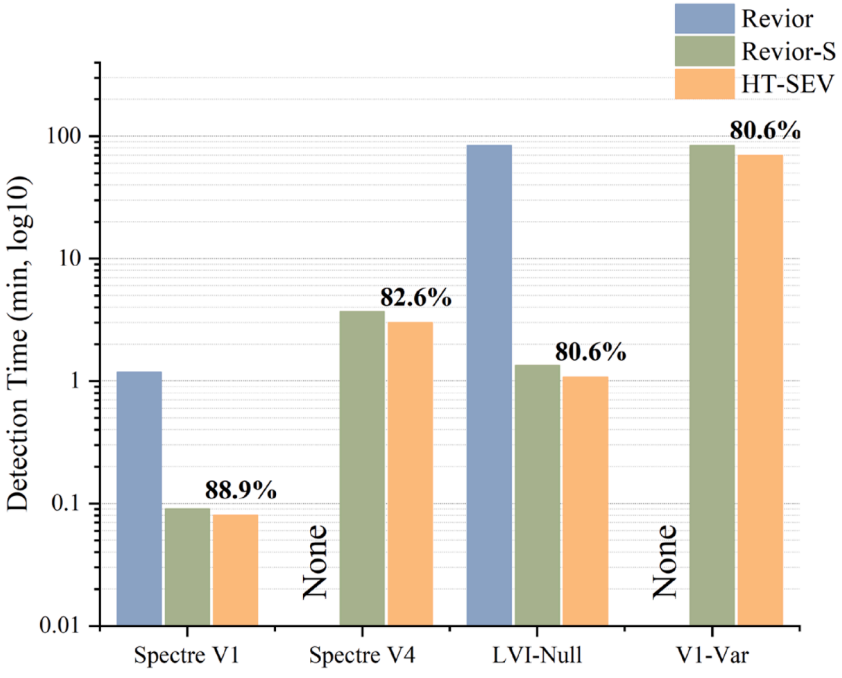

Fig. 12. Detection time. The four percentages represent the proportion of HT-SEV compared to Revizor-S.

Table 4
Valid Input Percentage. A valid input is defined as at least one other input that generates the same contract trace.

Method	Revizor	Revizor-s	HT-SEV
Percentage	4.30 %	99.97 %	100.00 %

(Section 3.5). HT-SEV-B is the comparison method that replaces bidirectional gradient mutation with random mutation. Compared to random mutation, bidirectional gradient mutation increases performance by an average of 8 %. Bidirectional gradient mutation quickly leads to 100 % coverage of the test program. Meanwhile, bidirectional gradient mutation diversifies the hardware trace and facilitates the detection of speculative violations. In contrast, random mutation is unstable and cannot test all code.

5. Discussions

5.1. Impact of each phase in testing

HT-SEV primarily introduces two key innovations: data dependency-driven test program generation and bidirectional mutation. These

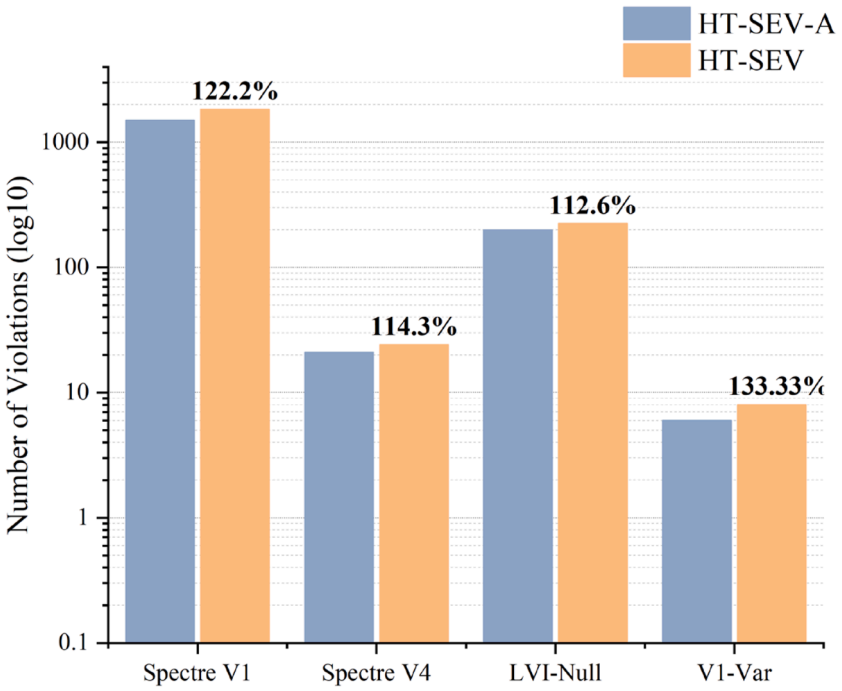

Fig. 13. Ablation Experiment 1 - Data Dependency-Driven Test Program Generation. The four percentages represent the proportion of HT-SEV compared to HT-SEV-A.

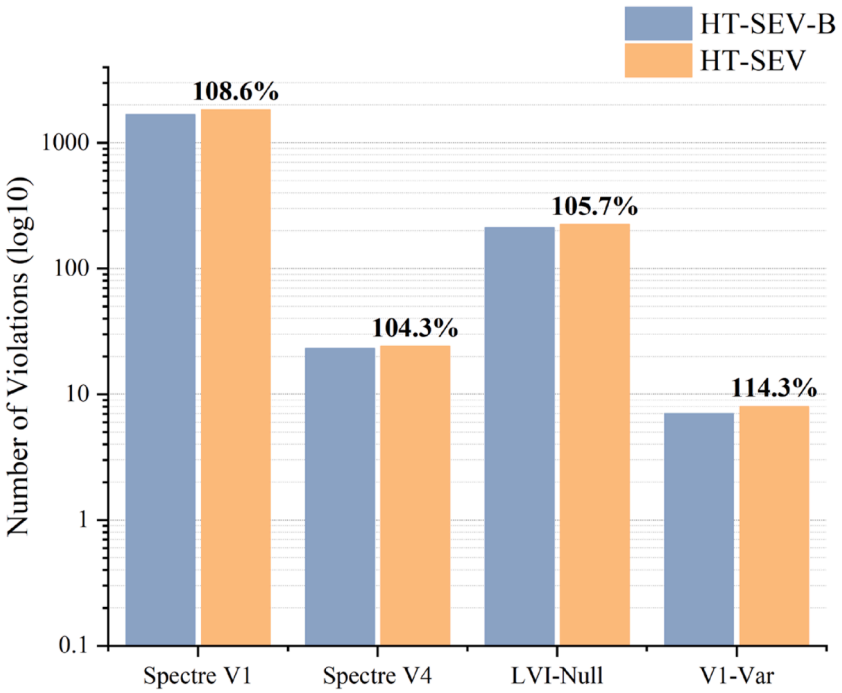

Fig. 14. Ablation Experiment 2 - Violation Analyze. The four percentages represent the proportion of HT-SEV compared to HT-SEV-B.

innovations address the issue of inadequate data dependency and fixed hardware trace. Experiment 1 and Experiment 2 demonstrate that the increase of HT-SEV in the number of violations detected is larger than the detection time. The reason is that HT-SEV generates a larger number of test programs that pass both filters than the compared methods, and processing these programs requires additional time. Meanwhile, ablation experiments (Experiment 4 and Experiment 5) reveal that data dependency-driven test program generation contributes more to performance increase than bidirectional gradient mutation. Therefore, the main point of speculative leak testing is to generate test programs that can trigger speculative execution, which is the main innovation of HT-SEV.

5.2. *Analysis of uncovered vulnerabilities*

As known from the paper (Oleksenko et al., 2023) and the experiments in this paper, the test program is the key factor. Due to the inherent randomness of the test program generation process, certain

vulnerabilities that satisfy the testing requirements are not detected, such as Spectre-BTB (Kocher et al., 2019) and Meltdown (Lipp et al., 2018). For instance, Spectre-BTB requires mistraining the BTB (Branch Target Buffer, BTB), while Meltdown requires page faults during testing. Generating test programs that meet these specific requirements is inherently challenging. Furthermore, HT-SEV cannot detect FPVI (Ragab et al., 2021) (Floating Point Value Injection, FPVI) vulnerability that rely on floating-point or SIMD (Single Instruction Multiple Data, SIMD) instructions because the test framework lacks certain x86–64 instructions. Additionally, detecting Meltdown vulnerability requires the capability to handle page faults, which is unavailable in the current test framework. However, this randomness is also essential for uncovering unknown speculative execution vulnerabilities. Balancing these competing factors of the test program generation phase is the key to solving such challenges.

5.3. Conversion to actual vulnerabilities/attacks

From the perspective of vulnerability identification, all vulnerabilities detected by HT-SEV can be exploited practically. Theoretically, HT-SEV utilizes the cache-side channel attack to collect cache access traces, replicating its behavior in real-world attack scenarios. Moreover, HT-SEV effectively minimizes noise interference during the testing process, preventing the occurrence of false positives in the results. In designing the test framework, HT-SEV strives to create a consistent microarchitectural state context for each test, thereby minimizing the influence of microarchitectural state changes on the test outcomes. In multiple tests of a single test program, the framework filters and discards cache access traces that appear only once, further reducing noise interference. Consequently, the violations detected by HT-SEV represent authentic speculative execution vulnerabilities within the tested environment. In practice, the actual availability of speculative execution vulnerabilities is determined by specific microarchitectural details. Since different processor have different architectures and defense measures, the threat scope of different speculative execution vulnerabilities differs. If HT-SEV successfully detect a vulnerability, it signifies that the vulnerability poses a real threat to the target processor without additional protection measures.

5.4. False positives and false negatives

In this manuscript and Model-based Relational Testing, there are several reasons for not applying false positives as evaluation metrics:

(1) Vulnerability Determination Logic.

The vulnerability determination logic of HT-SEV is based on whether an attack has successfully stolen information. HT-SEV determines vulnerability by detecting discrepancies in the multiple cache access traces (hardware traces) of a test program. If such a discrepancy exists, an attacker can exploit it to introduce private data into the cache and subsequently steal the data using a cache side-channel attack. Such a discrepancy detected by HT-SEV implies that the corresponding attack can be successfully carried out. Therefore, theoretically, HT-SEV does not produce false positives.

(2) Technical Measures to Eliminate Noise Interference.

In the testing framework, HT-SEV utilizes several measures to eliminate noise interference. These include:

- Microarchitectural state reset: Each test is initialized with a similar microarchitectural context to eliminate interference caused by microarchitectural noise;
- Noise filtering: Cache access traces that appear only once during executions of the same test program are eliminated, further reducing the impact of external noise.

(3) Validation of existing experimental results.

In the experiments of HT-SEV, Paper (Oleksenko et al., 2022) and Paper (Oleksenko et al., 2023), no false positives are detected. Even in the small number of false positives mentioned in Paper (Oleksenko et al., 2023), these “false positives” only come from the initial screening phase of the filter. False positives in the filter phase merely allow more test programs to the test phase and do not influence the final vulnerability determination. The final vulnerability determination depends solely on the cache access traces of the test program. These scenarios only reduce test efficiency.

For false negatives, the following limitations apply

(1) Limitations on the characterization of speculative execution vulnerabilities.

A false negative represents a successful attack that is not detected by HT-SEV. Based on the previous description of the vulnerability determination logic in false positives, such occurrences are not possible. If HT-SEV doesn’t detect a vulnerability, it means the attack is not successful.

If a false negative corresponds to a scenario where a code segment capable of triggering a vulnerability is executed, the task shifts to identifying whether the test program contains the code segment that can trigger the vulnerability. Identifying such code segments becomes to the detection of speculative execution attack gadgets in software. Related work in this area can be found in (Qi et al., 2021) and (Lu et al., 2025). Conversely, HT-SEV focuses on microarchitectural features, making the presence of speculative execution gadgets irrelevant to its core testing scope.

(2) Practical Limitations.

Consistent with Paper (Oleksenko et al., 2022) and Paper (Oleksenko et al., 2023), HT-SEV generates 100,000 test programs during testing. Manually determining whether these programs contain speculative execution attack gadgets is time-consuming and impractical.

Other Architectures and Vulnerability Variants

Given the widespread adoption of the x86 architecture in diverse fields, this manuscript and several related works have primarily focused on x86. Based on the theoretical foundation and implementation process of the proposed method, HT-SEV can be extended to other instruction set architectures. Such extensions require specific modifications in the implementation, particularly in test program generation (to accommodate different instruction sets), simulation execution, and information collection processes.

The detectability of new vulnerabilities/variants depends on architectural details. HT-SEV combines data dependency and fuzzing tests to increase the trigger rate of speculative execution and vulnerabilities, and exploits cache-side channel attacks to extract leakage information. This detection pattern aligns with the actual attack process of most speculative execution vulnerabilities, enabling HT-SEV to cover a broad spectrum of vulnerability types. However, speculative execution vulnerabilities have strong correlations with architectural type and detail. Some speculative execution vulnerabilities require a specific detection environment. For example:

- Spectre V1 (CVE-2017–5753): affect various mainstream processors from Intel and AMD;
- Spectre NG (CVE-2018–3665): limited to Intel processors;
- Foreshaow (CVE-2018–3615): only affect Intel 8th generation Coffee Lake architecture processors, other Intel processors are unaffected;
- INCEPTION (CVE-2023–20,569): limited to AMD processors.
- Zenbleed (CVE-2023–20,563): only affect AMD Zen2 processors, other AMD processors are unaffected.

Additionally, Analysis of Uncovered Vulnerabilities mentions that HT-SEV cannot detect certain specific vulnerabilities, such as Meltdown, Spectre-BTB and FPVI. This limitation stems from the model-based relational testing framework employed by the method.

In summary, HT-SEV is applicable to most speculative execution vulnerability types.

5.5. Automatic identification of vulnerability types

In Experiment 1, both Revizor and HT-SEV detected several thousand of speculative violations. While existing testing frameworks can minimize the test programs and input sequences that trigger leaks, these violations still require manual efforts to identify the types of vulnerabilities. This process is not only labor-intensive but also time-consuming. Future advancements in speculative leak testing should incorporate a module capable of automatically classifying these leaks.

6. Conclusions

This manuscript proposes a HT-SEV method whose core idea is to strengthen the data dependency between code lines and diversify the microarchitectural information (i.e., hardware trace). Data dependencies not only satisfy dependency relationships between codes, but also facilitate the construction of execution waiting scenarios. From the attacker's perspective, data dependency serves as the critical factor for accessing private data at different addresses. Consequently, this manuscript proposes a Data Dependency-Driven Test Program Generation technique, which constructs a register selected model based on the data flow and the real-time register distribution information (including frequency, step size and register first occurrence information). The register selected model guides the establishment of data dependencies between code lines, thereby increases the probability of triggering speculative execution. Traditional fuzzy testing typically improves test coverage through multiple tests, execution feedback, and complex mutation strategies. Since detecting microarchitectural speculative execution vulnerabilities requires generating a large number of test programs with simple structure and few instructions, this traditional complex mutation method is inefficient. To address issues of detection information convergence and insufficient adequacy, HT-SEV proposes an efficient bidirectional gradient mutation method. This method constructs the correlation between input variables and execution paths by analyzing fine-grained execution information. Furthermore, it explores critical conditions for path transitions to guide input mutations, achieving high path coverage and diverse detection information.

4 comparison experiments and 2 ablation experiments are conducted on 4 common speculative execution vulnerabilities. The comparison experiments involve percentage of valid test programs, number of violation instances, detection time, and the percentage of valid inputs. In Exp. 2 Number of Violations, this manuscript uses the number of detected violations instead of the regular false positives and false negatives, as discussed in Section 5. The experimental results demonstrate that the proposed method outperforms the state-of-the-art correlation method in all metrics. Specifically, it improves the number of detected violations by approximately 26 % and detection efficiency by 17 %. HT-SEV significantly improves the triggering rate of speculative execution and speculative execution vulnerabilities, and quickly achieves high coverage of testing programs and diversity of detection information. The ablation experiments further substantiate the effectiveness of data dependency-driven test program generation and bidirectional gradient mutation, improving the number of detected violations by roughly 20 % and 8 %, respectively.

HT-SEV boosts the trigger rate of speculative execution through data dependencies and implements cache-side channel attacks to detect data leaks during speculative execution. This pattern is consistent with the underlying principle of speculative execution vulnerabilities and speculative execution attacks, enabling HT-SEV to detect most of vulnerabilities. As mentioned in Section 5 Discussion, HT-SEV fails to detect a small number of vulnerabilities due to the specific characteristics of these vulnerabilities and the limitations of the testing framework. Future research will aim to address these uncovered speculative execution vulnerabilities.

CRediT authorship contribution statement

Chuan Lu: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Senlin Luo:** Writing – review & editing, Project administration, Methodology, Conceptualization. **Limin Pan:** Writing – review & editing, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work is supported in part by the Information Security Software Project (2020) of the Ministry of Industry and Information Technology, PR China under Grant CEIEC-2020-ZM02–0134.

Supplementary materials

Supplementary material associated with this article can be found, in the online version, at doi:10.1016/j.cose.2025.104567.

Appendix

A. Speculative Execution Vulnerabilities with CVE Numbers

This manuscript enumerates speculative execution vulnerabilities with CVE numbers in Table 5. In most cases, speculative execution attacks must be combined with other techniques to extract private information. This creates an opportunity to collect hardware traces using cache-side channel attacks. However, a small subset of speculative execution vulnerabilities cannot directly leverage cache-side channel attacks. These vulnerabilities are also limited to specific processor architectures. For instance, the Zenbleed vulnerability is exclusive to processors with the AMD Zen2 architecture ().

Table 5
Speculative execution vulnerabilities with CVE numbers.

No.	Name	Side channel/leakage mode	CVE	Affected	
				Intel	AMD
1	Spectre v1	cache	CVE-2017–5753	✓	✓
2	Spectre v2	cache	CVE-2017–5715	✓	✓
3	Meltdown	cache	CVE-2017–5754	✓	×

(*continued on next page*)

Table 5 (*continued*)

No.	Name	Side channel/leakage mode	CVE	Affected	
				Intel	AMD
4	SpectreRSB/ret2spec	cache	CVE-2018–15,572	✓	✓
5	Spectre-NG v3a	cache	CVE-2018–3640	✓	×
6	Spectre-NG v4	cache	CVE-2018–3639	✓	✓
7	Foreshadow	cache	CVE-2018–3615	✓	×
8	Spectre-NG	cache	CVE-2018–3665	✓	×
9	Spectre-NG v1.1	cache	CVE-2018–3693	✓	✓
10	Foreshadow-OS	cache	CVE-2018–3620	✓	×
11	Foreshadow-VMM	cache	CVE-2018–3646	✓	×
12	ZombieLoad	cache + buffer	CVE-2018–12,130	✓	×
13	Load Port	cache + buffer	CVE-2018–12,127	✓	×
14	RIDL	cache + buffer	CVE-2019–11,091	✓	×
15	Fallout	cache + buffer	CVE-2018–12,126	✓	×
16	SWAPGS	cache	CVE-2019–1125	✓	×
17	ZombieLoad v2	cache + buffer	CVE-2019–11,135	✓	×
18	CacheOut	cache	CVE-2020–0549	✓	×
19	RIDL	cache + buffer	CVE-2020–0548	✓	×
20	Load Value Injection (LVI)	cache + buffer	CVE-2020–0551	✓	×
21	CROSSTalk	cache + buffer	CVE-2020–0543	✓	×
22	Floating Point Value Injection	cache	CVE-2021–0086	✓	×
23	Floating Point Value Injection	cache	CVE-2021–26,314	×	✓
24	Speculative Code Store Bypass	cache	CVE-2021–0089	✓	×
25	Speculative Code Store Bypass	cache	CVE-2021–26,313	×	✓
26	Branch History Injection	cache	CVE-2022–0001	✓	×
27	Branch History Injection	cache	CVE-2022–0002	✓	×
28	Shared Buffers Data Read	direct steal	CVE-2022–21,123	✓	×
29	Shared Buffers Data Sampling	cache + buffer	CVE-2022–21,125	✓	×
30	Shared Buffers Data Sampling	cache + buffer	CVE-2022–21,127	✓	×
31	Device Register Partial Write	direct steal	CVE-2022–21,166	✓	×
32	Phantom	cache	CVE-2022–23,825	×	✓
33	Retbleed	cache	CVE-2022–29,900	×	✓
34	Retbleed	cache	CVE-2022–29,901	✓	×
35	Cross-Thread Return Address Predictions	cache	CVE-2022–27,672	×	✓
36	Zenbleed	direct steal	CVE-2023–20,593	×	✓
37	Inception	cache	CVE-2023–20,569	×	✓
38	Downfall	cache	CVE-2022–40,982	✓	×

B. Data Source

Each instruction is categorized into different subsets depending on its type, as shown in Table 6. To satisfy the basic functionality of the test program, each instruction subset also contains 8 basic instructions (ADC, ADD, CMP, DEC, INC, NEG, SBB, SUB). Except for system instructions (e.g., SYSCALL), instructions incorrectly emulated by Unicorn (e.g., ROR), and control-flow instructions that are not contractually available (RET, CALL, and indirect jumps), which are not yet supported by the test framework, these x86 instruction subsets make up a complete base x86–64 instruction set.

Table 6
Instruction subset.

Name	Type	Instructions
cond	conditional branches	ADC, ADD, CMP, DEC, INC, NEG, SBB, SUB, J*, LOOP*, JMP (unconditional direct jump)
strn	string operations	ADC, ADD, CMP, DEC, INC, NEG, SBB, SUB, CLC, CLD, CMC, LAHF, LOCK, REPE, REPNE, SAHF, SCASB, SCASD, SCASW, STC, STD
dmul	division and multiplication	ADC, ADD, CMP, DEC, INC, NEG, SBB, SUB, DIV, IMUL, MUL
flag	operations on flags	ADC, ADD, CMP, DEC, INC, NEG, SBB, SUB, CLC, CLD, CMC, LAHF, SAHF, STC, STD
lock	atomics with LOCK prefix	ADC, ADD, CMP, DEC, INC, NEG, SBB, SUB, LOCK ADC, LOCK ADD, LOCK CMP, LOCK DEC, LOCK INC, LOCK NEG, LOCK SBB, LOCK SUB, LOCK BSF, LOCK BSR, LOCK BT, LOCK BTC, LOCK BTR, LOCK BTS, LOCK NOT, LOCK OR, LOCK TEST, LOCK XOR
atom	atomics without LOCK prefix	ADC, ADD, CMP, DEC, INC, NEG, SBB, SUB, CMPXCHG, XADD, LOCK CMPXCHG, LOCK XADD
dxfr	data transfer	ADC, ADD, CMP, DEC, INC, NEG, SBB, SUB, BSWAP, MOV, MOVSX, MOVZX, XCHG
setc	conditional byte set	ADC, ADD, CMP, DEC, INC, NEG, SBB, SUB, SET*
nop	NOP instructions	ADC, ADD, CMP, DEC, INC, NEG, SBB, SUB, NOP
logi	logical operations	ADC, ADD, CMP, DEC, INC, NEG, SBB, SUB, AND, NOT, OR, TEST, XOR
conv	data type conversion	ADC, ADD, CMP, DEC, INC, NEG, SBB, SUB, CBW, CDQ, CWD, CWDE
cmov	conditional moves	ADC, ADD, CMP, DEC, INC, NEG, SBB, SUB, CMOV*
bit	bit test and bit scan	ADC, ADD, CMP, DEC, INC, NEG, SBB, SUB, BSF, BSR, BT, BTC, BTR, BTS

Data availability

Data will be made available on request.

References

Buiras, P., Nemati, H., Lindner, A., Guanciale, R., 2021. Validation of side-channel models via observation refinement. 54th Annu. IEEE/ACM Int. Symp. Microarchitecture 578–591. https://doi.org/10.1145/3466752.3480130.

Clifton Nick. SPECTRE Variant 1 scanning tool. https://access.redhat.com/blogs/766093/posts/3510331. 2018 (accessed 17 April 2025).
Daniel, L.A., Bardin, S., Rezk, T., 2020. Binsec/rel: efficient relational symbolic execution for constant-time at binary-level. IEEE Symp. Secur. Priv. 1021–1038. https://doi.org/10.1109/SP40000.2020.00074.
Daniel, L.A., Bardin, S., Rezk, T., 2021. Hunting the haunter-efficient relational symbolic execution for spectre with haunted relse. Netw. Distrib. Syst. Secur. https://doi.org/10.14722/ndss.2021.24286.
Disselkoen, C., Kohlbrenner, D., Porter, L., Tullsen, D., 2017. Prime+Abort: a timer-free high-precision L3 cache attack using Intel TSX. 26th USENIX Secur. Symp. 51–67. https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/disselkoen.
Downfall, M., 2023. Downfall: exploiting speculative data gathering. In: 32nd USENIX Security Symposium, pp. 7179–7193. https://www.usenix.org/conference/usenixsecurity23/presentation/moghimi.
Fabian, X., Guarnieri, M., Patrignani, M., 2022. Automatic detection of speculative execution combinations. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, pp. 965–978. https://doi.org/10.1145/3548606.3560555.
Fadiheh, M.R., Stoffel, D., Barrett, C., Mitra, S., Kunz, W., 2019. Processor hardware security vulnerabilities and their detection by unique program execution checking. In: 2019 Design, Automation & Test in Europe Conference & Exhibition (DATE), pp. 994–999. https://doi.org/10.23919/DATE.2019.8715004.
Ghaniyoun, M., Barber, K., Zhang, Y., Teodorescu, R., 2021. Introspectre: a pre-silicon framework for discovery and analysis of transient execution vulnerabilities. 2021 ACM/IEEE 48th Annu. Int. Symp. Comput. Archit. (ISCA) 874–887. https://doi.org/10.1109/ISCA52012.2021.00073.
Gras, B., Giuffrida, C., Kurth, M., et al., 2020. ABSynthe: automatic blackbox side-channel synthesis on commodity microarchitectures. Netw. Distrib. Syst. Secur. Symp. https://doi.org/10.14722/ndss.2020.23018.
Gruss, D., Spreitzer, R., Mangard, S., 2015. Cache template attacks: automating attacks on inclusive last-level caches. 24th USENIX Secur. Symp. 897–912. https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/gruss.
Gruss, D., Maurice, C., Wagner, K., Mangard, S., 2016. Flush+Flush: a fast and stealthy cache attack. Detect. Intrusions Malware Vulnerability Assess. 279–299. https://doi.org/10.1007/978-3-319-40667-1_14.
Guarnieri, M., Köpf, B., Reineke, J., Vila, P., 2021. Hardware-software contracts for secure speculation. In: IEEE Symposium on Security and Privacy, pp. 1868–1883. https://doi.org/10.1109/SP40001.2021.00036.
Hofmann, J., Vannacci, E., Fournet, C., Köpf, B., Oleksenko, O., 2023. Speculation at fault: modeling and testing microarchitectural leakage of CPU exceptions. USENIX Secur. Symp. 7143–7160. https://www.usenix.org/conference/usenixsecurity23/presentation/hofmann.
Hur, J., Song, S., Kim, S., et al., 2022. Specdoctor: differential fuzz testing to find transient execution vulnerabilities. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, pp. 1473–1487. https://doi.org/10.1145/3548606.3560578.
Johannesmeyer, B., Koschel, J., Razavi, K., et al., 2022. Kasper: scanning for generalized transient execution gadgets in the Linux kernel. Netw. Distrib. Syst. Secur. Symp. https://doi.org/10.14722/ndss.2022.24221.
Kim, J., van Schaik, S., Genkin, D., Yarom, Y., 2023. Ileakage: browser-based timerless speculative execution attacks on apple devices. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, pp. 2038–2052. https://doi.org/10.1145/3576915.3616611.
Kocher, P., Horn, J., Fogh, A., Genkin, D., Gruss, D., Haas, W., et al., 2019. Spectre attacks: exploiting speculative execution. In: 2019 IEEE Symposium on Security and Privacy (SP), pp. 1–19. https://doi.org/10.1145/3399742.
Li, C., Gaudiot, J.L, 2021. Detecting spectre attacks using hardware performance counters. IEEE Trans. Comput. 71 (6), 1320–1331. https://doi.org/10.1109/TC.2021.3082471.
Lin F., Wang Z., Sasaki H. Teapot: efficiently uncovering spectre gadgets in COTS binaries. https://arxiv.org/abs/2411.11624. 2024 (accessed 17 April 2025).
Lipp, M., Schwarz, M., Gruss, D., Prescher, T., Haas, W., Horn, J., et al., 2018. Meltdown: reading kernel memory from user space. In: 27th USENIX Security Symposium, pp. 973–990. https://doi.org/10.1145/3357033.
Lu, J., Ma, G., Zhang, G., 2024. Fuzzy machine learning: a comprehensive framework and systematic review. IEEE Trans. Fuzzy Syst. 32 (7), 3861–3878. https://doi.org/10.1109/TFUZZ.2024.3387429.
Lu, C., Luo, S., Pan, L., 2025. SCR-Spectre: spectre gadget detection method with strengthened context relevance. Comput. Electr. Eng. 123, 110029. https://doi.org/10.1016/j.compeleceng.2024.110029.
Moghimi, D., Lipp, M., Sunar, B., Schwarz, M., 2020. Medusa: microarchitectural data leakage via automated attack synthesis. 29th USENIX Secur. Symp. 1427–1444. https://www.usenix.org/conference/usenixsecurity20/presentation/moghimi-medusa.
Nemati, H., Buiras, P., Lindner, A., Guanciale, R., Jacobs, S., 2020. Validation of abstract side-channel models for computer architectures. Comput. Aided Verif. 225–248. https://doi.org/10.1007/978-3-030-53288-8_12.
Oleksenko, O., Trach, B., Silberstein, M., et al., 2020. SpecFuzz: bringing spectre-type vulnerabilities to the surface. 29th USENIX Secur. Symp. 1481–1498. https://www.usenix.org/conference/usenixsecurity20/presentation/oleksenko.
Oleksenko, O., Fetzer, C., Köpf, B., Silberstein, M., 2022. Revizor: testing black-box CPUs against speculation contracts. In: Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, pp. 226–239. https://doi.org/10.1145/3503222.3507729.
Oleksenko, O., Guarnieri, M., Köpf, B., Siberstein, M., 2023. Hide and seek with spectres: efficient discovery of speculative information leaks with random testing. In: IEEE Symposium on Security and Privacy, pp. 1737–1752. https://doi.org/10.1109/SP46215.2023.10179391.
Open Source Security Inc. Respectre: the State of the art in spectre defenses. https://grsecurity.net/respectre_announce. 2018 (accessed 17 April 2025).
Osvik, D.A., Shamir, A., Tromer, E., 2006. Cache attacks and countermeasures: the case of AES. Top. Cryptol.–CT-RSA 2006 1–20. https://doi.org/10.1007/11605805_1.
Pardoe Andrew. Spectre mitigations in MSVC. https://devblogs.microsoft.com/cppblog/spectre-mitigations-in-msvc/. 2018 (accessed 17 April 2025).
Ponce-de-León, H., Kinder, J., 2022. Cats vs. spectre: an axiomatic approach to modeling speculative execution attacks. IEEE Symp. Secur. Priv. 235–248. https://doi.org/10.1109/SP46214.2022.9833774.
Qi, Z., Feng, Q., Cheng, Y., et al., 2021. SpecTaint: speculative taint analysis for discovering spectre gadgets. Netw. Distrib. Syst. Secur. Symp. https://doi.org/10.14722/ndss.2021.23466.
Ragab, H., Barberis, E., Bos, H., Giuffrida, C., 2021. Rage against the machine clear: a systematic analysis of machine clears and their implications for transient execution attacks. 30th USENIX Secur. Symp. 1451–1468. https://www.usenix.org/conference/usenixsecurity21/presentation/ragab.
Rajapaksha, C., Delshadtehrani, L., Egele, M., et al., 2023. SIGFuzz: a framework for discovering microarchitectural timing side channels. In: Design, Automation & Test in Europe Conference & Exhibition, pp. 1–6. https://doi.org/10.23919/DATE56975.2023.10136966.
Rostami, M., Zeitouni, S., Kande, R., et al., 2024. Lost and found in speculation: hybrid speculative vulnerability detection. In: Proceedings of the 61st ACM/IEEE Design Automation Conference, pp. 1–6. https://doi.org/10.1145/3649329.3658469.
Tol, M.C., Gulmezoglu, B., Yurtseven, K., et al., 2021. Fastspec: scalable generation and detection of spectre gadgets using neural embeddings. IEEE Eur. Symp. Secur. Priv. 616–632. https://doi.org/10.1109/EuroSP51992.2021.00047.
Trippel, C., Lustig, D., Martonosi, M., 2018. Checkmate: automated synthesis of hardware exploits and security litmus tests. 2018 51st Annu. IEEE/ACM Int. Symp. Microarchitecture 947–960. https://doi.org/10.1109/MICRO.2018.00081.
Tromer, E., Osvik, D.A., Shamir, A., 2010. Efficient cache attacks on AES, and countermeasures. J. Cryptol. 23, 37–71. https://doi.org/10.1007/s00145-009-9049-y.
Wang, G., Chattopadhyay, S., Gotovchits, I., et al., 2019. oo7: low-overhead defense against spectre attacks via program analysis. IEEE Trans. Softw. Eng. 47 (11), 2504–2519. https://doi.org/10.1109/TSE.2019.2953709.
Wang, Q., Tang, M., Xu, K., et al., 2025. Unveiling and evaluating vulnerabilities in branch predictors via a three-step modeling methodology. ACM Trans. Archit. Code Optim. 22 (1), 1–26. https://doi.org/10.1145/3711923.
Weber, D., Ibrahim, A., Nemati, H., et al., 2021. Osiris: automated discovery of microarchitectural side channels. 30th USENIX Secur. Symp. 1415–1432. https://www.usenix.org/conference/usenixsecurity21/presentation/weber.
Xiao, Y., Zhang, Y., Teodorescu, R., 2020. SPEECHMINER: a framework for investigating and measuring speculative execution vulnerabilities. NDDS Symp. 2020. https://doi.org/10.14722/ndss.2020.23105.
Yarom, Y., Falkner, K., 2014. FLUSH+RELOAD: a high resolution, low noise, l3 cache side-channel attack. 23rd USENIX Secur. Symp. 719–732. https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/yarom.
Zhu, X., Wen, S., Camtepe, S., Xiang, Y., 2022. Fuzzing: a survey for roadmap. ACM Comput. Surv. 54 (11s), 1–36. https://doi.org/10.1145/3512345.